

Pattern Matching and Machine Learning for Audio Signal Processing

Prof. Dr. Frank Kurth

Prof. Dr. Michael Clausen

INSTITUTE OF COMPUTER SCIENCE, UNIVERSITY OF BONN

Chapter 5: Audio Matching

Chapter 5: Audio Matching	2
Audio matching: a motivating example	2
Audio Matching Problem	3
Feature design requirements	4
Feature design: synopsis	5
Subband decomposition: preliminaries	6
STFT revisited	7
The log-frequency spectrogram	8
Chroma features: preliminaries	9
Chroma	10
Chromagrams	11
The chromatic scale played on a piano	12
Spectrogram & spectral peaks vs. chromagrams	13
Logarithmic compression	14
Logarithmic compression: examples	15
Timbre robustness of chroma features	16
Robust chroma normalization	17
Quantization	18
Smoothing, downsampling & CENS	19
Chroma features (Bernstein)	20
CENS features (Bernstein)	21
Chroma features (Bernstein vs. Sawallisch)	22
CENS features (Bernstein vs. Sawallisch)	23
Diagonal Audio Matching	24
Global distance to query	25
Visualization of top seven hits	26
Global tempo variations (1)	27
Global tempo variations (2)	28
Global tempo variations (3)	29
Global tempo variations (4)	30

Dynamic time warping: preliminaries	31
DTW: introductory example *	32
DTW: task specification	33
Warping paths	34
Dynamic programming: preliminaries	35
Dynamic programming algorithm.	36
Computing an optimal warping path	37
DTW computations: an example.	38
Optimal local alignments	39
DTW-based audio matching.	40
References	41

Audio matching: a motivating example

Different versions of Beethoven's Fifth (first 20 measures)

- Bernstein 
- Sawallisch 
- Karajan 
- Scherbakov 

Summary:

- The Bernstein and the Sawallisch interpretation are pretty similar.
- Karajan chose a much faster tempo.
- Scherbakov played Franz Liszt's piano transcription of Beethoven's Fifth.

Audio Matching Problem

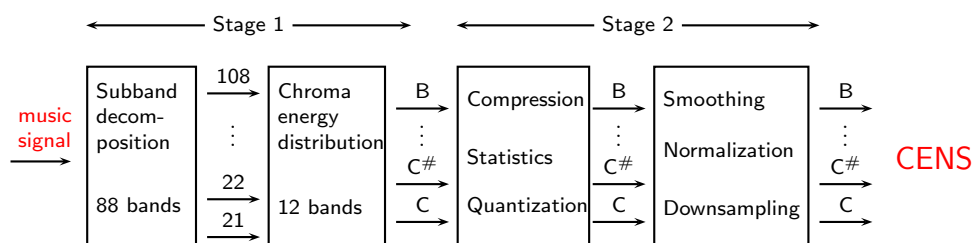
- **Given:** Large music database containing several
 - ◆ recordings of the same piece of music,
 - ◆ interpretations by various musicians,
 - ◆ arrangements in different instrumentations.
- **Goal:** Given a short **query audio clip** identify in the database all corresponding audio clips of similar musical content
 - ◆ irrespective of the specific interpretation,
 - ◆ irrespective of the instrumentation,
 - ◆ automatically and efficiently.
- For this matching task, audio fingerprints based on **spectral peaks are quite inconsistent**
 - ◆ with regard to their **frequency stamps**: due to differences in timbre,
 - ◆ with regard to their **time stamps**: due to differences in tempo.

Feature design requirements

The audio matching task calls for features that are more flexible than spectral peaks. The new features to be designed should be **robust** with regard to differences in

- **timbre** (example: orchestral version vs. piano transcription)
- **dynamics**
- **global and local tempo**
- **articulation**
- **accentuation**
- **execution of note groups** (example: grace notes, trills, arpeggios).

Feature design: synopsis



- Stage 1: Local chroma energy distribution features

- Stage 2: Normalized short-time statistics

⇒ **CENS** = **C**hroma **E**nergy **N**ormalized **S**tatistics

Subband decomposition: preliminaries

- We describe how to decompose a music signal into **frequency bands** corresponding to musical notes and start with preliminary remarks.
- We assume a **twelve-tone equal-tempered scale**, as used for Western music. The MIDI **pitch** $p \in [0 : 127]$ corresponds to the **center frequency** (in Hertz)

$$F_{\text{pitch}}(p) = 2^{\frac{p-69}{12}} \cdot 440 .$$

- In particular, $p = 69$ corresponds to 440 Hz, the **concert pitch** A4. Furthermore, as each octave consists of 12 semitones, we see that a transition from p to $p + 12$ leads—as requested—to a doubling in frequency

$$F_{\text{pitch}}(p + 12) = 2 \cdot F_{\text{pitch}}(p) .$$

- The 88 keys of a piano correspond to the MIDI pitches $p \in [21 : 108]$. In the following we limit ourselves to these pitches.

STFT revisited

We briefly recall the short time Fourier transform to fix the notation.

- Let $x : \mathbb{Z} \rightarrow \mathbb{R}$ be a real-valued discrete signal obtained by equidistant sampling with respect to a fixed **sampling rate** F_s given in Hertz.
- The STFT $\mathcal{X}$ of x with respect to a fixed real-valued **window** $w \in \mathbb{R}^N$ and a fixed **hop size** $H \in [1 : N]$ is given by

$$\mathcal{X}(n, k) = \sum_{\ell=0}^{N-1} x(\ell + nH) \cdot w(\ell) \cdot \exp(-2\pi i k \ell / N)$$

with $n \in \mathbb{Z}$ and $k \in [0 : K]$, where $K = N/2$ is the frequency index corresponding to the **Nyquist frequency**.

- Recall from Chapter 2 that each Fourier coefficient $\mathcal{X}(n, k)$ is associated with the following **physical time position** T_{coef} (given in seconds) and with the **physical frequency** F_{coef} (given in Hertz):

$$T_{\text{coef}}(n) := T_{\text{coef}}(n|H, F_s) = nH/F_s \quad \text{and} \quad F_{\text{coef}}(k) := F_{\text{coef}}(k|N, F_s) = kF_s/N .$$

The log-frequency spectrogram

- The logarithmic perception of frequency motivates the use of a time-frequency representation with a logarithmic frequency axis labeled by the pitches of the equal-tempered scale.
- **Basic idea:** Assign each spectral coefficient $\mathcal{X}(n, k)$ to the pitch with center frequency that is closest to the frequency $F_{\text{coef}}(k) = kF_s/N$. We define for each pitch $p \in [0 : 127]$ via the **cutoff frequencies** $F_{\text{pitch}}(p \pm 0.5)$ the set

$$P(p) := P(p|N, F_s) = \{k \mid F_{\text{pitch}}(p - 0.5) \leq F_{\text{coef}}(k) < F_{\text{pitch}}(p + 0.5)\}.$$

- From this we obtain a **log-frequency spectrogram** $\mathcal{X}_{\text{LF}} : \mathbb{Z} \times [0 : 127] \rightarrow \mathbb{R}_{\geq 0}$ defined by

$$\mathcal{X}_{\text{LF}}(n, p) := \sum_{k \in P(p)} |\mathcal{X}(n, k)|^2.$$

- **Example:** If $N = 2^{12}$ and $F_s = 44100$, then $P(69|N, F_s) = \{40, 41, 42\}$. Thus

$$\mathcal{X}_{\text{LF}}(n, 69) = |\mathcal{X}(n, 40)|^2 + |\mathcal{X}(n, 41)|^2 + |\mathcal{X}(n, 42)|^2.$$

Chroma features: preliminaries

- Playing a single note on an instrument may result in a complex sound that contains a mixture of different frequencies.
- Such a musical tone can be described as a superposition of pure sinusoids. A **partial** is any of the sinusoids by which a musical tone is described.
- The frequency of the lowest partial is called the **fundamental frequency** of the sound. The **pitch** of a musical tone is usually determined by the fundamental frequency.
- A **harmonic** (or a **harmonic partial**) is a partial that is an integer multiple of the fundamental frequency. All harmonics different from the fundamental frequency are also called **overtone**s.
- Partials, as well as harmonics, are counted upwards along the frequency axis. The fundamental frequency is the **first partial**, as well as the **first harmonic** of a musical tone.
- The timbre of a sound strongly relates to the energy distribution in the harmonics. Therefore, due to the octave equivalence, **chroma features** show a high degree of **robustness to variations in timbre**.

Chroma

- The **human perception of pitch is periodic** in the sense that two pitches are perceived as **similar in “color”** if they differ by octaves.
- More formally, two pitches $p, q \in [0 : 127]$ are **octave equivalent** iff

$$p \equiv q \bmod 12.$$

- This induces a partitioning of the pitches in 12 **pitch classes**, also called **chroma**.
- For $c \in [0 : 11]$ the corresponding chroma consists of all pitches $p \in [0 : 127]$ whose remainder mod 12 equals c . It is common practice to use instead of remainders the following names for pitch classes:

remainder	0	1	2	3	4	5	6	7	8	9	10	11
chroma	C	C [#]	D	D [#]	E	F	F [#]	G	G [#]	A	A [#]	B

- **Example:** $C = \{0, 12, 24, 32, 48, 60, 72, 84, 96, 108, 120\}$.
- As C4 (=middle C) has MIDI pitch $p = 60$ and $60 \bmod 12 = 0$, the above nomenclature is compatible with the MIDI pitches.

Chromagrams

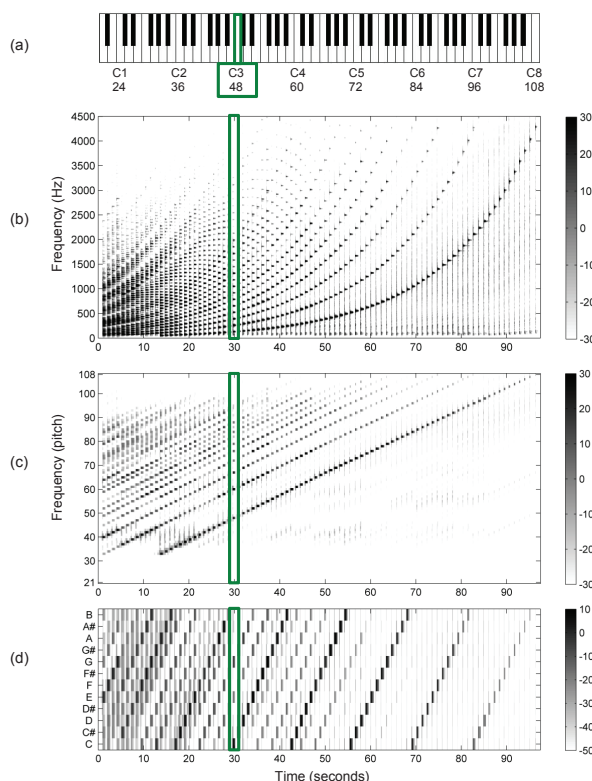
- Chroma features aggregate all spectral information that relates to a given pitch class into a single coefficient.
- Given a pitch-based log-frequency spectrogram $\mathcal{X}_{\text{LF}} : \mathbb{Z} \times [0 : 127] \rightarrow \mathbb{R}_{\geq 0}$, the corresponding **chromagram** $\mathcal{C} : \mathbb{Z} \times [0 : 11] \rightarrow \mathbb{R}_{\geq 0}$ is defined by

$$\mathcal{C}(n, c) := \sum_{p \in [0:127]: p \bmod 12 = c} \mathcal{X}_{\text{LF}}(n, p).$$

- More explicitly:

$$\mathcal{C}(n, c) = \begin{cases} \sum_{k=0}^{10} \mathcal{X}_{\text{LF}}(n, c + 12 \cdot k) & \text{if } c \in [0 : 7], \\ \sum_{k=0}^9 \mathcal{X}_{\text{LF}}(n, c + 12 \cdot k) & \text{if } c \in [8 : 11]. \end{cases}$$

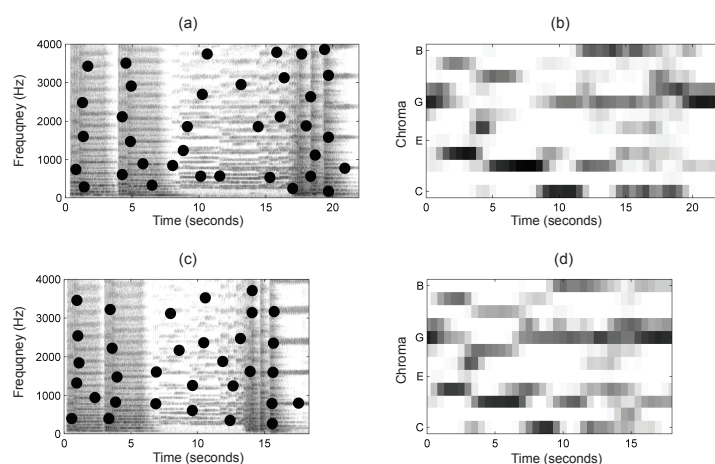
The chromatic scale played on a piano



- (a) piano keys representing the chromatic scale ranging from A0 ($p = 21$) to C8 ($p = 108$),
- (b) magnitude spectrum,
- (c) log-frequency spectrogram,
- (d) chromagram.
- The note C3 played at time $t = 30$ has been highlighted.

Spectrogram & spectral peaks vs. chromagrams

In the next figure we compare different representations derived from recordings of the very beginning of Beethoven's Fifth Symphony.



- (a) Spectrogram and spectral peaks for a Bernstein recording.
- (b) Chromagram for a Bernstein recording.
- (c) Spectrogram and spectral peaks for a Karajan recording.
- (d) Chromagram for a Karajan recording.

Summary: Spectral peaks are not designed to handle musical variations. The two chromagrams exhibit a much higher degree of similarity than the corresponding spectrograms, thus revealing a much higher robustness against performance variations.

Logarithmic compression

- The problem with representations such as a spectrogram or a chromagram is that its values possess a **large dynamic range**. As a result, **small, but still relevant values may be dominated or obscured by large values**. Therefore, the idea of **compression** is to balance out this discrepancy by reducing the difference between large and small values with the effect of enhancing the small values.
- **Logarithmic compression**, an alternative to using a decibel scale, works as follows: Let $\gamma > 0$ be a positive constant and $\Gamma_\gamma : \mathbb{R}_{>0} \rightarrow \mathbb{R}_{>0}$ defined by

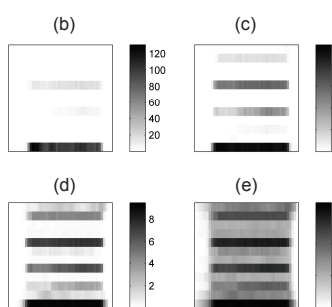
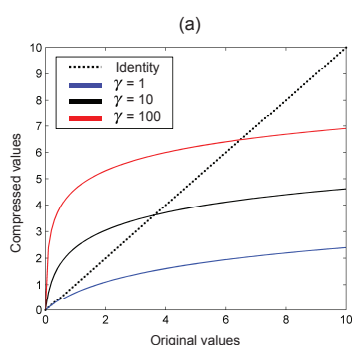
$$\Gamma_\gamma(\nu) := \log(1 + \gamma \cdot \nu).$$

- Hence, as opposed to the dB function, the function Γ_γ yields a positive value $\Gamma_\gamma(\nu)$ for **any** positive ν . Now, for a representation with positive values such as a spectrogram, log-frequency spectrogram or chromagram, one obtains a compressed version by applying the function Γ_γ to each of the values.
- For example, the **compressed version of the chromagram** $\mathcal{C}$ is the concatenation $\Gamma_\gamma \circ \mathcal{C}$ defined by

$$(\Gamma_\gamma \circ \mathcal{C})(n, c) := \log(1 + \gamma \cdot \mathcal{C}(n, c)).$$

Logarithmic compression: examples

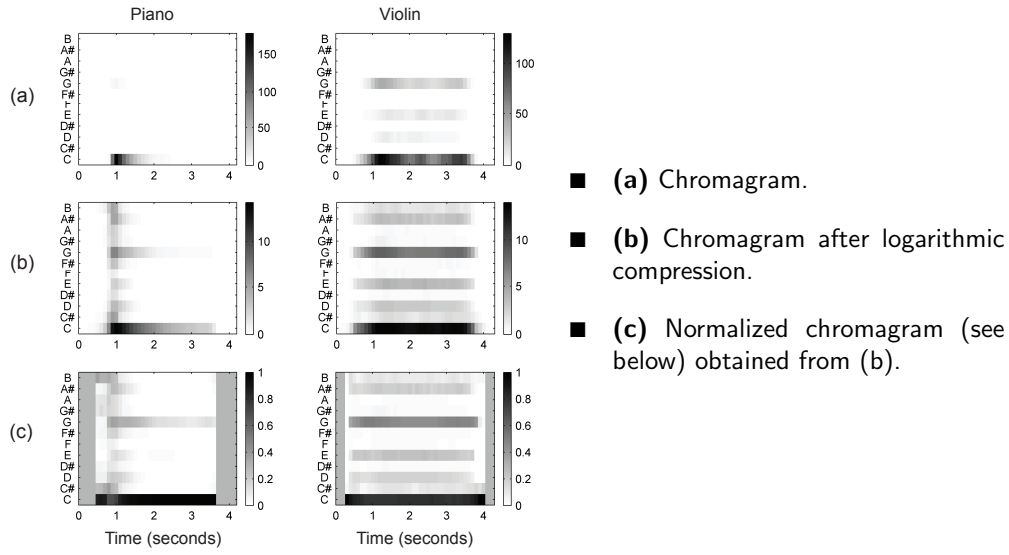
We illustrate the role of compression to reduce the difference between large and small values with the effect of enhancing the small values.



- (a) Compression function Γ_γ for different γ .
- (b) Chromagram for the note C4 played by a piano.
- (c) Compressed chromagram using $\gamma = 1$.
- (d) Compressed chromagram using $\gamma = 100$.
- (e) Compressed chromagram using $\gamma = 100000$.

Timbre robustness of chroma features

We illustrate various (compressed versions of) chroma representations for the note C4 played by a piano and a violin.



- This indicates that chroma features are robust to differences in timbre ... at least to a certain degree.

Robust chroma normalization

- To make the chromagrams also **robust to changes in dynamics**, one can normalize the features with respect to some suitable **norm** $\|\cdot\|$. To this end, each of the twelve-dimensional chroma vectors $x = \mathcal{C}(n, \cdot) \in \mathbb{R}^{12}$ for a frame index $n \in \mathbb{Z}$ (n th column of $\mathcal{C}$) is replaced by the vector $x/\|x\|$.
- We will use the **Euclidean norm (2-norm)** $\|x\|_2 := (\sum_{i=0}^{11} |x(i)|^2)^{1/2}$ and the **Manhattan norm (1-norm)** $\|x\|_1 := \sum_{i=0}^{11} |x(i)|$.
- The normalization procedure is only possible if $\|x\| > 0$. Also for very small values $\|x\|$ that may occur in passages of silence before the actual start of the recording or during long pauses, normalization would lead to more or less random—and therefore meaningless—chroma value distributions. Therefore, if $\|x\|$ falls below a certain threshold, the vector x may be replaced by a uniform vector of norm one instead of dividing by $\|x\|$.
- Mathematically, this normalization procedure can be described as follows: Let $\mathbb{S}^{11} \subset \mathbb{R}^{12}$ be the unit sphere containing all 12-dimensional vectors of norm one. Then, for a given threshold $\varepsilon > 0$, we define a projection operator $\pi^\varepsilon : \mathbb{R}^{12} \rightarrow \mathbb{S}^{11}$ by $\pi^\varepsilon(x) := x/\|x\|$, if $\|x\| > \varepsilon$, otherwise $\pi^\varepsilon(x) := (1, \dots, 1)^\top / \|(1, \dots, 1)^\top\|$.
- Based on this operator, each chroma vector x is replaced by $\pi^\varepsilon(x)$.

Quantization

- To further reduce the effect of local fluctuations due to variations in local tempo, articulation, and note execution we apply three additional procedures: **quantization**, **smoothing**, and **downsampling**.
- Let $X = (x_1, x_2, \dots, x_N)$ denote the resulting sequence of 1-normalized chroma vectors $x_n \in [0, 1]^{12}$, $n \in [1 : N]$. Next, we define a **quantization function** $Q : [0, 1] \rightarrow [0 : 4]$ by

$a \in$	$[0, 0.05)$	$[0.05, 0.1)$	$[0.1, 0.2)$	$[0.2, 0.4)$	$[0.4, 1]$
$Q(a)$	0	1	2	3	4

- Note that the thresholds are chosen in a logarithmic fashion to account for the **logarithmic perception of sound intensity**.
- **Step 1:** we quantize each chroma vector $x_n = (x_n(0), \dots, x_n(11))^T \in [0, 1]^{12}$ by applying Q to each component of x_n , yielding

$$Q(x_n) := (Q(x_n(0)), \dots, Q(x_n(11)))^T.$$

- The quantization procedure results in the **quantized sequence**

$$(Q(x_1), \dots, Q(x_N)).$$

Smoothing, downsampling & CENS

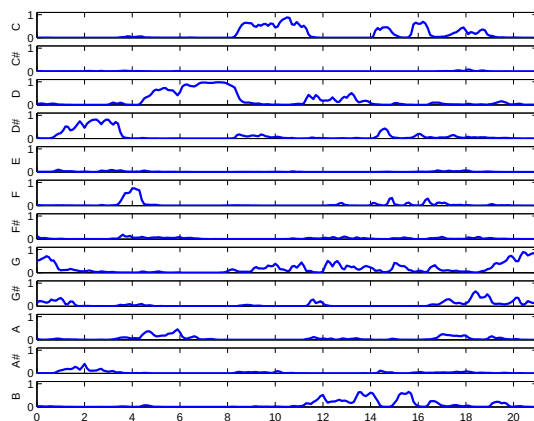
- **Step 2:** the quantized sequence $(Q(x_1), \dots, Q(x_N))$ is further **smoothed**. To this end, we fix a number $\ell \in \mathbb{N}$ that determines the length of a smoothing window (e.g., a Hann window) and then consider local averages (weighted by the window function) of each of the twelve components of the sequence $(Q(x_1), \dots, Q(x_N))$. This again results in a sequence of 12-dimensional vectors with nonnegative entries.
- **Step 3:** this sequence is **downsampled** by a factor of d , and the resulting vectors are normalized with respect to the Euclidean norm.
- The two steps, quantization and smoothing, amount to computing a weighted statistics of the energy distribution over a window of ℓ consecutive vectors. Therefore, we call the resulting features

$$\text{CENS}_d^\ell$$

(chroma energy normalized statistics).

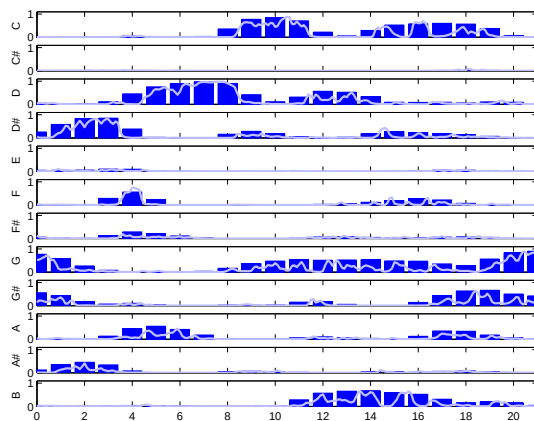
- The main idea of **CENS features** is that taking statistics over relatively large windows smooths out local deviations in tempo, articulation, and execution of note groups such as trills or arpeggios.

Chroma features (Bernstein)



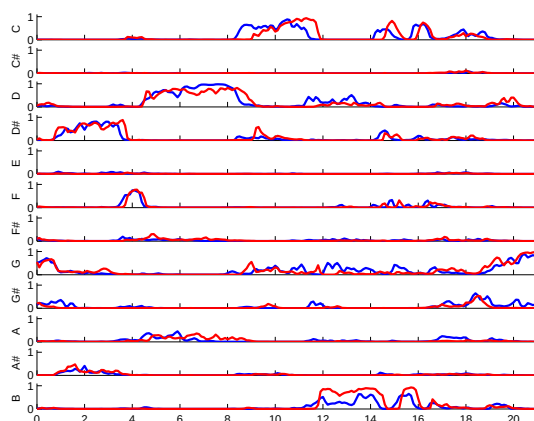
- **Beethoven's Fifth:**
Bernstein
- **Resolution:**
10 features/second
- **Window:**
200 milliseconds.

CENS features (Bernstein)



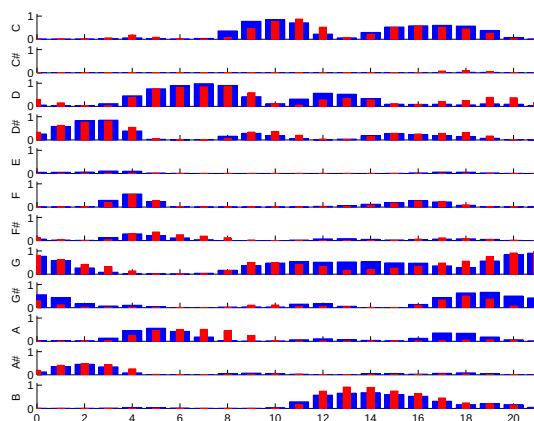
- **Beethoven's Fifth:**
Bernstein
- **CENS feature:**
 $CENS_{10}^{41}$
- **Resolution:**
1 feature/second
- **Statistics window:**
4100 milliseconds.

Chroma features (Bernstein vs. Sawallisch)



- **Beethoven's Fifth:**
Bernstein vs. Sawallisch
- **Resolution:**
10 features/second
- **Window:**
200 milliseconds.

CENS features (Bernstein vs. Sawallisch)

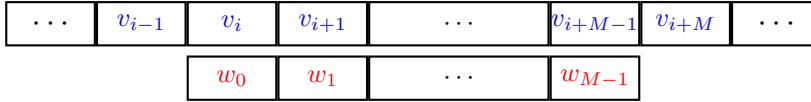


- **Beethoven's Fifth:**
Bernstein vs. Sawallisch
- **CENS feature:**
 $CENS_{10}^{41}$
- **Resolution:**
1 feature/second
- **Statistics window:**
4100 milliseconds.

Diagonal Audio Matching

Computation of CENS feature sequences:

- **Database:** $D \rightsquigarrow \text{CENS}_{10}^{41}[D] = (v_0, \dots, v_{N-1})$
- **Query:** $Q \rightsquigarrow \text{CENS}_{10}^{41}[Q] = (w_0, \dots, w_{M-1})$
- **Example:** $N \approx 500000$, $M \approx 20$



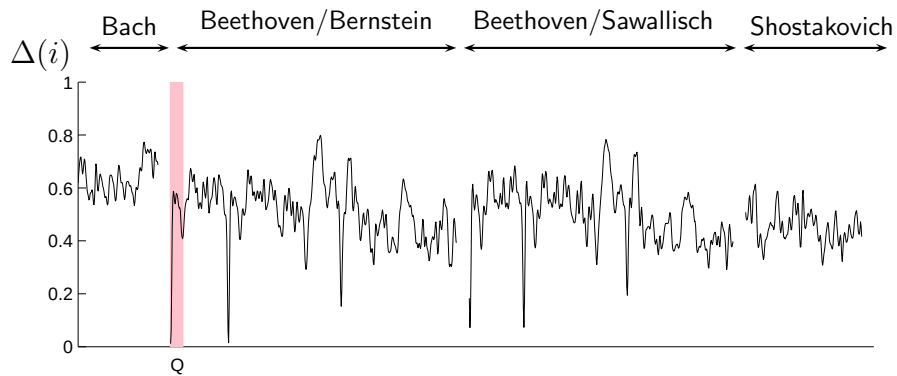
- Local distance of $(v_i, v_{i+1}, \dots, v_{i+M-1})$ and $(w_0, w_1, \dots, w_{M-1})$:

$$\Delta(i) := 1 - \frac{1}{M} \sum_{m=0}^{M-1} \langle v_{i+m} | w_m \rangle$$

- leads to a global distance function $\Delta : [0 : N - M] \rightarrow [0, 1]$.

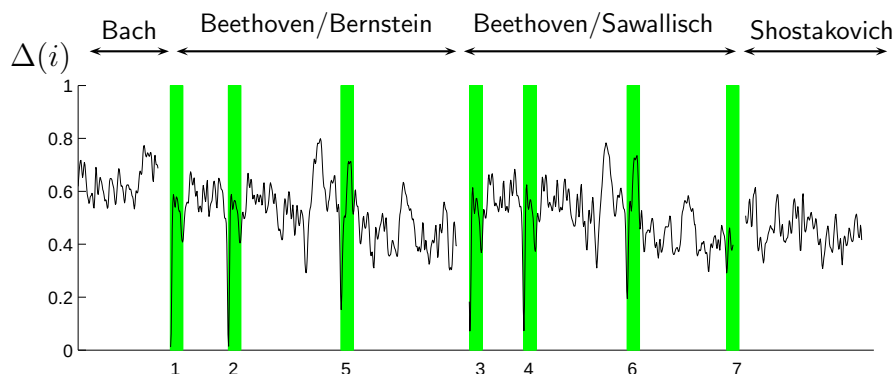
Global distance to query

Query: Beethoven's Fifth/Bernstein, first 20 seconds



Visualization of top seven hits

Query: Beethoven's Fifth/Bernstein, first 20 seconds



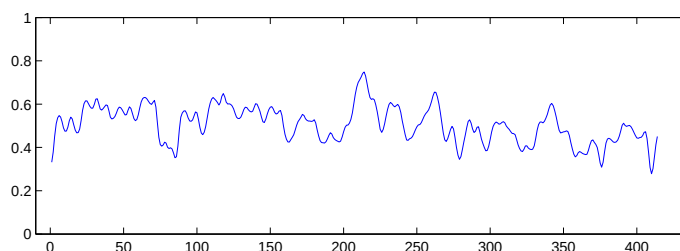
- First movement of Beethoven's Fifth is based on the **sonata form**.
- Hit # 1 corresponds to the query (first theme), # 2 to the repetition of this theme, and # 5 to a slightly modified version of it in the recapitulation.
- Similarly: hits # 3, # 4, # 6.

Global tempo variations (1)

Query: Beethoven's Fifth/Bernstein, first 20 Seconds

Problem: Karajan is much faster $\rightsquigarrow \Delta$ w.r.t. $\text{CENS}_{10}^{41}[Q]$ useless

Solution?

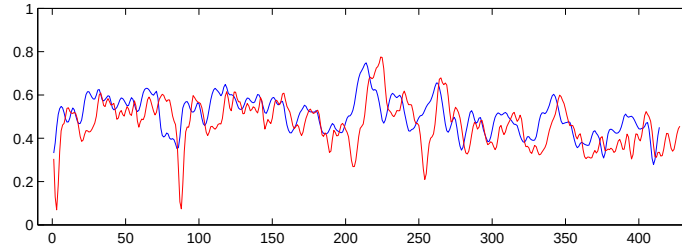


Global tempo variations (2)

Query: Beethoven's Fifth/Bernstein, first 20 Seconds

Problem: Karajan is much faster $\rightsquigarrow \Delta$ w.r.t. $\text{CENS}_{10}^{41}[Q]$ useless

Solution: Make Bernstein's query faster at (inexpensive!) feature level (see Exercises)
& compute new Δ , e.g., $\text{CENS}_{13}^{53}[Q]$

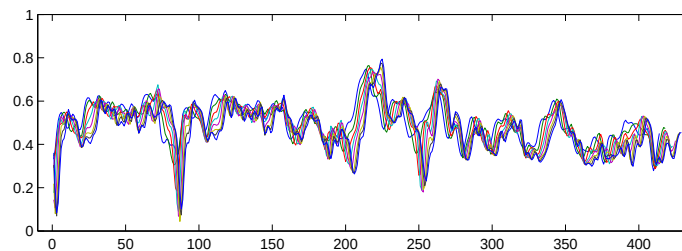


Global tempo variations (3)

Query: Beethoven's Fifth/Bernstein, first 20 Seconds

Problem: Karajan is much faster $\rightsquigarrow \Delta$ w.r.t. $\text{CENS}_{10}^{41}[Q]$ useless

Solution: Compute Δ for several tempi, e.g., $\text{CENS}_{d_j}^{\ell_j}[Q]$, for $j \in [1 : 8]$.

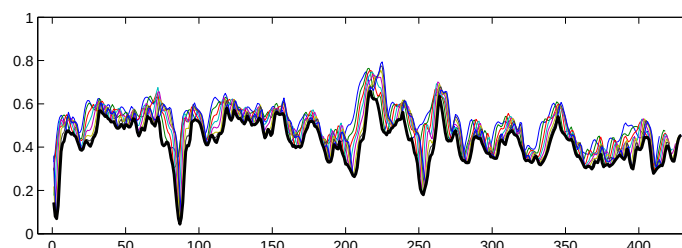


Global tempo variations (4)

Query: Beethoven's Fifth/Bernstein, first 20 Seconds

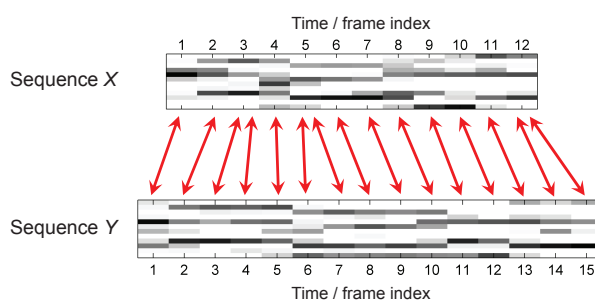
Problem: Karajan is much faster $\rightsquigarrow \Delta$ w.r.t. $\text{CENS}_{10}^{41}[Q]$ useless

Solution: Minimum over all resulting Δ 's $\rightsquigarrow \Delta^{\min}$



Dynamic time warping: preliminaries

- We sketch another, more flexible approach to audio matching. This approach is closely related to **music synchronization**, whose goal is to automatically link the various positions in one representation of a piece of music with the corresponding positions within another representation.
- The task of music synchronization amounts to time align two feature sequences in an optimal way. A standard technique for solving this task is known as **dynamic time warping** (DTW).



DTW: introductory example *

- The objective of DTW is to compare two given sequences.
- In the music synchronization scenario, these two sequences are, for example, chroma representations of two different versions of the same piece of music. Let

$$X = (x_1, x_2, \dots, x_N) \quad \text{and} \quad Y = (y_1, y_2, \dots, y_M)$$

be two sequences of chroma vectors x_n and y_m . For example, X may be a chroma vector sequence obtained from an orchestral version of Beethoven's Fifth and Y a sequence obtained from a piano version. Since the orchestral performance underlying the sequence X is played faster than the piano performance underlying the sequence Y , the two sequences do not have the same length even though they musically correspond to each other.

- The goal of DTW is to compensate for differences in tempo by finding a possibly nonlinear alignment between the elements of the two sequences. Intuitively speaking, this can be achieved by either skipping certain elements of a sequence or by using certain elements more than once.
- For example, in the alignment shown in the last figure, the element x_3 is assigned to the two elements y_3 and y_4 . Similarly, the elements x_5 and x_{12} are used twice in the overall alignment.

DTW: task specification

- Let $\mathcal{F}$ denote a **feature space**.
 - ◆ Chroma example: $\mathcal{F} = \{x \in [0, 1]^{12} \mid \|x\|_2 = 1\}$.
- $c : \mathcal{F} \times \mathcal{F} \rightarrow \mathbb{R}_{\geq 0}$ a **local cost/distance measure**.
 - ◆ Chroma example: $c(x, y) = 1 - \langle x | y \rangle$ (cosine measure).
- $X \in \mathcal{F}^N$ and $Y \in \mathcal{F}^M$ **feature sequences**.
- $\mathbf{C} \in \mathbb{R}^{N \times M}$, defined by $C(n, m) := c(x_n, y_m)$, the **cost matrix**.
- $P = (p_1, \dots, p_L)$, an **(N, M) -warping path** (of length L), specifies the global alignment (the red arrows on Slide 31), where $p_\ell = (n_\ell, m_\ell) \in [1 : N] \times [1 : M]$, for more details see the next slide.
- $c_P(X, Y) := \sum_{\ell=1}^L c(x_{n_\ell}, y_{m_\ell}) = \sum_{\ell=1}^L \mathbf{C}(n_\ell, m_\ell)$ denotes the **total cost** of the warping path P between X and Y .
- **Goal:** compute a cost-minimizing (N, M) -warping path P^* and $\text{DTW}(X, Y) := c_{P^*}(X, Y)$, the **DTW-distance** between X and Y .

Warping paths

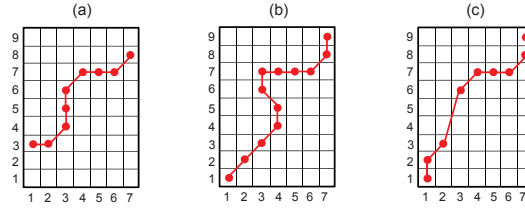
A warping path models a global alignment between two feature sequences. More formally:

Definition 1. $P = (p_1, \dots, p_L)$ is an (N, M) -warping path of length L between the elements of the sequences $X \in \mathcal{F}^N$ and $Y \in \mathcal{F}^M$ if the sequence of index pairs $p_\ell = (n_\ell, m_\ell) \in [1 : N] \times [1 : M]$ fulfills the following constraints:

- **Boundary condition:** $p_1 = (1, 1)$ and $p_L = (N, M)$.
- **Monotonicity condition:** $n_1 \leq n_2 \leq \dots \leq n_L$ and $m_1 \leq m_2 \leq \dots \leq m_L$.
- **Step size condition:** $p_{\ell+1} - p_\ell \in \{(1, 0), (0, 1), (1, 1)\}$ for $\ell \in [1 : L - 1]$.

The following figure illustrates violations of the warping path conditions:

(a) boundary condition, (b) monotonicity condition, (c) step size condition.



Dynamic programming: preliminaries

- We determine an optimal warping path P^* by an $O(NM)$ algorithm that is based on **dynamic programming**. The general idea behind dynamic programming is to break down a given problem into simpler subproblems and then to combine the solutions of the subproblems to reach an overall solution. In the case of DTW, the idea is to derive an optimal warping path for the original sequences from optimal warping paths for truncated subsequences. This idea can then be applied recursively.
- To formalize this idea, we define the prefix sequences $X(1 : n) := (x_1, \dots, x_n)$ for $n \in [1 : N]$ and $Y(1 : m) := (y_1, \dots, y_m)$ for $m \in [1 : M]$ and set

$$\mathbf{D}(n, m) := \text{DTW}(X(1 : n), Y(1 : m)).$$

- The values $\mathbf{D}(n, m)$ define an $N \times M$ matrix $\mathbf{D}$, the **accumulated cost matrix**. Each value $\mathbf{D}(n, m)$ specifies the total (or accumulated) cost of an optimal warping path starting at cell $(1, 1)$ and ending at cell (n, m) .
- Obviously, one has $\mathbf{D}(N, M) = \text{DTW}(X, Y)$.

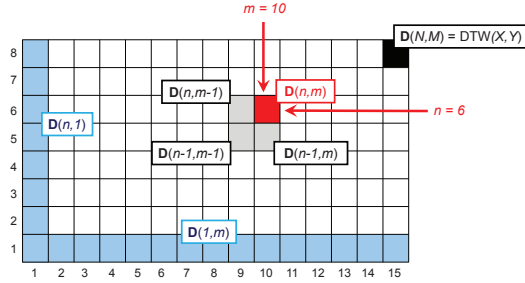
Dynamic programming algorithm

The next equations show how the accumulated cost matrix $\mathbf{D}$ can be computed recursively: by

$$\mathbf{D}(n, 1) = \sum_{k=1}^n \mathbf{C}(k, 1) \quad \text{and} \quad \mathbf{D}(1, m) = \sum_{k=1}^m \mathbf{C}(1, k)$$

for $n \in [1 : N]$ and $m \in [1 : M]$, and for $n \in [2 : N]$ and $m \in [2 : M]$ by

$$\mathbf{D}(n, m) = \mathbf{C}(n, m) + \min(\mathbf{D}(n-1, m-1), \mathbf{D}(n-1, m), \mathbf{D}(n, m-1)). \quad (1)$$



- Illustration of the recursive computation of the DTW distance.
- The blue cells indicate the entries $\mathbf{D}(n, 1)$ and $\mathbf{D}(1, m)$.
- The black cell indicates the final result $\mathbf{D}(N, M)$.
- The red cell indicates the current entry $\mathbf{D}(n, m)$ being computed by Eq. (1).

Computing an optimal warping path

- An optimal warping path $P^* = ((1, 1) = p_1, p_2, \dots, p_L = (N, M))$ will be computed backwards, $P^* = (q_L, q_{L-1}, \dots, q_1)$, via the following procedure:
 - Put $\ell = 1$ and $q_\ell = (N, M)$.
 - Repeat until $q_\ell = (1, 1)$:
 - ◆ Increase ℓ by one. Let $(n, m) = q_{\ell-1}$.
 - ◆ If $n = 1$, then $q_\ell = (1, m-1)$,
 - ◆ else if $m = 1$, then $q_\ell = (n-1, m)$,
 - ◆ else $q_\ell = \operatorname{argmin} \{\mathbf{D}(n-1, m-1), \mathbf{D}(n-1, m), \mathbf{D}(n, m-1)\}$.
(If argmin is not unique, take lexicographically smallest cell.)
 - Set $L = \ell$ and return $P^* = (q_L, q_{L-1}, \dots, q_1)$.
- A formal proof of the optimality of P^* is left as an exercise.

DTW computations: an example

- **Feature space:** $\mathcal{F} = \mathbb{R}$.
- **Local cost measure:** $c(x, y) := |x - y|$, for $x, y \in \mathbb{R}$.
- **Feature sequences:** $X = (1, 3, 3, 8, 1)$ and $Y = (2, 0, 0, 8, 7, 2)$.

(a)

	1	1	1	7	6	1
∞	6	8	8	0	1	6
∞	1	3	3	5	4	1
∞	1	3	3	5	4	1
∞	1	1	1	7	6	1
	2	0	0	8	7	2

(b)

	10	10	11	14	13	9
∞	9	11	13	7	8	14
∞	3	5	7	10	12	13
∞	2	4	6	8	12	13
∞	1	2	3	10	16	17
	2	0	0	8	7	2

(c)

1	3	3	8	1	
↑	↑	↑	↑	↑	
2	0	0	8	7	2

- (a) Cost matrix $\mathbf{C}$ for the two sequences X and Y over $\mathcal{F} = \mathbb{R}$ using the absolute differences as the local cost measure.
- (b) Accumulated cost matrix $\mathbf{D}$ with an optimal warping path.
- (c) Resulting alignment.

Optimal local alignments

- So far, we have discussed **global DTW distances**. However, audio matching ask for **optimal local alignments**.
- We are going to specify the task of computing optimal local alignments.
- Let $X \in \mathcal{F}^N$ and $Y \in \mathcal{F}^M$ be two feature sequences.
 - ◆ Audio matching scenario: X corresponds to a short query, Y to the database. Thus $N \ll M$.
- For $1 \leq a \leq b \leq M$ we use the notation $Y(a : b) := (y_a, y_{a+1}, \dots, y_b)$.
- Then the audio matching task is to determine the indices defined by

$$(a^*, b^*) := \operatorname{argmin}_{(a,b): 1 \leq a \leq b \leq M} \operatorname{DTW}(X, Y(a : b)).$$

- Surprisingly, with a slight modification of the original DTW algorithm one can solve this task.

DTW-based audio matching

- We define a modified accumulated cost matrix $\mathbf{D}$: for $n \in [1 : N]$ and $m \in [1 : M]$ put

$$\mathbf{D}(n, 1) = \sum_{k=1}^n \mathbf{C}(k, 1) \quad \text{and} \quad \mathbf{D}(1, m) = \mathbf{C}(1, m).$$

The remaining values of $\mathbf{D}$ are computed as in Eq. (1) on Slide 36:

$$\mathbf{D}(n, m) = \mathbf{C}(n, m) + \min(\mathbf{D}(n-1, m-1), \mathbf{D}(n-1, m), \mathbf{D}(n, m-1)).$$

- Then $b^* = \operatorname{argmin}_{b \in [1:M]} D(N, b)$.
- To determine a^* , use the above backtracking procedure and start with $q_1 = (N, b^*)$ (instead of (N, M)) and stop as soon as the first row of $\mathbf{D}$ is reached by some element $q_L = (1, m)$, $m \in [1 : M]$ (instead of $q_L = (1, 1)$).
- Then $a^* := m$.
- Furthermore, $(q_L, q_{L-1}, \dots, q_1)$ defines an optimal warping path between X and the subsequence $Y(a^* : b^*)$.
- We refer to the Exercises for more details.

References

To a large extent this chapter is based on Section 7.2 and Section 3.2 of the very recommendable book:

- **Meinard Müller: Fundamentals of Music Processing, Springer 2015.**

For alternative and more efficient approaches to audio matching using fault-tolerant indexing or locality sensitive hashing techniques we refer to

- Kurth & Müller: Efficient index-based audio matching, IEEE Transactions on Audio, Speech, and Language Processing, 16 (2008), pp. 382–395.
- Casey, Rhodes & Slaney: Analysis of minimum distances in high-dimensional musical spaces, IEEE Transactions on Audio, Speech, and Language Processing, 16 (2008), pp. 1015–1028.
- Grosche & Müller: Toward characteristic audio shingles for efficient cross-version music retrieval, in Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP), 2012, pp. 473–476.